Predictive Memory Leak Detection Using Time Series Analysis

Preliminary research draft | 14 September 2026 | Introduction only

1. Introduction

Memory leaks occur when software retains allocated memory that is no longer needed. In long-running applications, repeated retention can reduce available memory and contribute to performance degradation or failure. However, increasing memory use can also reflect legitimate workload changes or caching, so utilization alone cannot confirm a leak. Prior work has explored detecting leak-like growth from memory-utilization time series without inspecting an application's internal objects [1].

Operating-system telemetry provides a basis for studying this behavior over time. Memory measurements describe resource availability; CPU activity, page faults, swap activity and disk throughput can provide workload context when available and informative. Forecasting uses past measurements to estimate a future memory value. Assi et al. compared Box-Jenkins and long short-term memory (LSTM) models for Windows performance forecasting [2]. Their comparison motivates evaluating an LSTM against simpler alternatives before attributing practical value to model complexity.

Accurate forecasting and useful leak warnings are distinct objectives. A model may predict most memory values closely while missing a brief period of sustained growth. A sensitive growth rule may identify injected leaks while also triggering during ordinary activity. Forecast error alone cannot establish detection quality. This study therefore asks whether improved memory forecasts translate into better warnings, assessed through recognition of labeled leak intervals and alarms outside those intervals.

The implemented pipeline forecasts a memory-pressure proxy defined as one minus the ratio of available memory to total physical memory. This quantity describes memory availability; it is not an estimated probability of a leak. Persistence, local linear extrapolation and a restricted autoregressive integrated moving average (ARIMA) baseline are compared with a small multivariate LSTM. Each recording is divided chronologically into training, validation and test periods. Learned preprocessing parameters are fitted on training data, and model and warning settings are selected using validation data. Forecasts are evaluated using mean absolute error and root mean squared error; warning behavior is assessed separately.

The pilot combines two complementary experimental settings without merging their timelines. The Antarex HPC Fault Dataset supplies real telemetry collected under controlled fault injection, including labeled memory-leak intervals [3]-[5]. Its CPU-MEM_mono block supports forecasting and leak-warning evaluation. Westermo provides industrial test-system telemetry [6]; systems 1-3 are used as separate forecasting experiments. Their leak status is unknown, so alarms on Westermo cannot be scored as confirmed true or false leak detections.

The study contributes a reproducible pilot comparison, a separate evaluation of trend-only and forecast-assisted warning rules, and a local prototype for replaying the verified experiments. Conclusions are limited to the observed workloads and available labels. The experiments measure recognition after leak injection begins; they do not establish warning before memory exhaustion or prevention of system failure.

References

[1] A. Jindal, P. Staab, P. Kulkarni, J. Cardoso, M. Gerndt and V. Podolskiy, "Memory Leak Detection Algorithms in the Cloud-based Infrastructure," arXiv:2106.08938, 2021. https://doi.org/10.48550/arXiv.2106.08938.

[2] M. J. Assi, A. A. Fahad and B. Al-Sarray, "Time series forecast modeling for the Windows operating system performance using Box-Jenkins and LSTM models," International Journal of Nonlinear Analysis and Applications, vol. 13, no. 2, pp. 2121-2131, 2022. https://doi.org/10.22075/ijnaa.2022.26974.3505.

[3] A. Netti, Z. Kiziltan, O. Babaoglu, A. Sirbu, A. Bartolini and A. Borghesi, "Antarex HPC Fault Dataset," version 1.0, Zenodo, 2018. https://doi.org/10.5281/zenodo.2553224.

[4] A. Netti, Z. Kiziltan, O. Babaoglu, A. Sirbu, A. Bartolini and A. Borghesi, "FINJ: A Fault Injection Tool for HPC Systems," in Euro-Par 2018: Parallel Processing Workshops, Lecture Notes in Computer Science, vol. 11339, pp. 800-812, 2019. https://doi.org/10.1007/978-3-030-10549-5_62.

[5] A. Netti, Z. Kiziltan, O. Babaoglu, A. Sirbu, A. Bartolini and A. Borghesi, "Online Fault Classification in HPC Systems through Machine Learning," arXiv:1810.11208, 2018, revised 2019. https://doi.org/10.48550/arXiv.1810.11208.

[6] P. E. Strandberg and Y. Marklund, "The Westermo test system performance data set," arXiv:2311.14510, 2023. https://doi.org/10.48550/arXiv.2311.14510.